

Python for AI & Machine Learning

Class 16 – Abstraction in Python (Fourth Pillar of OOP)

Course: Python for AI & Machine Learning

Module: Object-Oriented Programming (OOP)

Class: 16

Duration: 2 Hours (Theory + Practical)

Learning Outcomes

By the end of this session, students will be able to:

- Define Abstraction and explain its importance.
 - Understand why Abstraction is one of the four pillars of OOP.
 - Differentiate between Abstraction and Encapsulation.
 - Create and use Abstract Classes.
 - Implement Abstract Methods using Python's abc module.
 - Build applications using Abstraction.
 - Apply Abstraction in AI and Machine Learning projects.
-

Module Recap

In the previous sessions, we learned:

- Class and Objects
- Constructors
- Instance Variables
- Class Variables
- Methods
- Encapsulation
- Inheritance
- Polymorphism

Today, we will complete the **Fourth Pillar of Object-Oriented Programming (OOP)**.

Four Pillars of Object-Oriented Programming

Pillar	Purpose
--------	---------

Encapsulation	Protects data from unauthorized access.
---------------	---

Inheritance	Allows code reuse through parent-child relationships.
-------------	---

Polymorphism	Enables one interface to have multiple implementations.
--------------	---

Abstraction	Hides implementation details and exposes only essential functionality.
--------------------	--

Introduction to Abstraction

Imagine driving a car.

As a driver, you only need to know:

- How to start the car
- How to accelerate
- How to brake
- How to steer

You **do not** need to understand:

- How the engine burns fuel
- How the transmission changes gears
- How the braking system works
- How sensors communicate with the engine

The internal complexity is hidden from you.

You only interact with the features you need.

This is **Abstraction**.

Definition of Abstraction

Abstraction is the process of **hiding the implementation details of a system and exposing only the essential features required by the user.**

Simple Definition

Abstraction means showing only what is necessary and hiding how it is implemented.

Why Do We Need Abstraction?

Suppose a software application contains thousands of lines of code.

If every user had to understand all of that code before using the application, it would be impossible to use.

Abstraction solves this problem by providing a simple interface while hiding the internal complexity.

Benefits

- Simplifies application usage.
 - Reduces complexity.
 - Improves security.
 - Makes maintenance easier.
 - Encourages modular programming.
 - Makes software scalable.
-

Real-World Examples

Example 1 – ATM Machine

When using an ATM, you can:

- Withdraw Cash
- Deposit Money
- Check Balance
- Transfer Money

You do not know:

- How the PIN is verified.
- How the bank server processes requests.
- How transactions are stored.
- How encryption is performed.

The implementation remains hidden.

This is **Abstraction**.

Example 2 – Mobile Phone

You can:

- Make Calls
- Browse the Internet
- Open Applications
- Send Messages

You do not know:

- Memory management
- Processor scheduling
- Signal processing
- Operating system internals

The complexity is hidden.

Example 3 – AI Chatbot

When you use ChatGPT or Gemini, you simply type a prompt and receive an answer.

Behind the scenes, the system performs:

- Tokenization

- Embedding generation
- Transformer computations
- Attention mechanisms
- Neural network inference
- GPU processing

Users never interact with these internal processes.

This is another example of Abstraction.

Characteristics of Abstraction

- Hides unnecessary implementation details.
- Shows only essential functionality.
- Improves user experience.
- Provides a standard interface.
- Encourages reusable code.
- Reduces software complexity.

Abstraction vs Encapsulation

Encapsulation

Protects data

Focuses on data security

Uses private/protected members

Restricts direct access

Abstraction

Hides implementation

Focuses on simplicity

Uses abstract classes and abstract methods

Exposes only required functionality

What is an Abstract Class?

An **Abstract Class** is a class that cannot be instantiated directly.

It acts as a blueprint for other classes.

Characteristics

- Cannot create objects.
 - Can contain abstract methods.
 - Can contain normal methods.
 - Must be inherited by another class.
-

The abc Module

Python provides the **abc (Abstract Base Class)** module to implement abstraction.

Import it using:

```
from abc import ABC, abstractmethod
```

Components

ABC

- Used to create an Abstract Class.

@abstractmethod

- Used to declare Abstract Methods.
-

What is an Abstract Method?

An **Abstract Method** is a method that has only a declaration and no implementation.

It defines what a child class **must** implement.

Example:

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
```

```
@abstractmethod
```

```
def sound(self):
```

```
    pass
```

Notice that sound() contains no implementation.

Implementing an Abstract Class

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
```

```
    @abstractmethod
```

```
    def sound(self):
```

```
        pass
```

```
class Dog(Animal):
```

```
    def sound(self):
```

```
        print("Dog Barks")
```

```
dog = Dog()
```

```
dog.sound()
```

Output

Dog Barks

Multiple Child Classes

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
```

```
    @abstractmethod
```

```
    def sound(self):
```

```
        pass
```

```
class Dog(Animal):
```

```
    def sound(self):
```

```
        print("Dog Barks")
```

```
class Cat(Animal):
```

```
    def sound(self):
```

```
        print("Cat Meows")
```

```
dog = Dog()
```

```
cat = Cat()
```



```
dog.sound()
```

```
cat.sound()
```

Output

Dog Barks

Cat Meows

What Happens If We Do Not Implement the Abstract Method?

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
```

```
    @abstractmethod
```

```
    def sound(self):
```

```
        pass
```

```
class Dog(Animal):
```

```
    pass
```

```
dog = Dog()
```

Output

TypeError:

Can't instantiate abstract class Dog

with abstract method sound

Python prevents object creation because the child class has not implemented all required abstract methods.

Abstract Class with Normal Methods

An abstract class can contain both abstract methods and normal methods.

```
from abc import ABC, abstractmethod
```

```
class Employee(ABC):
```

```
    @abstractmethod
```

```
    def salary(self):
```

```
        pass
```

```
    def company(self):
```

```
        print("Digix Technologies")
```

```
class Developer(Employee):
```

```
    def salary(self):
```

```
        print("Salary: ₹60,000")
```

```
developer = Developer()
```

```
developer.company()
```

```
developer.salary()
```

Output

Digix Technologies

Salary: ₹60,000

AI Example – Machine Learning Framework

```
from abc import ABC, abstractmethod
```

```
class MLModel(ABC):
```

```
    @abstractmethod
```

```
    def train(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def predict(self):
```

```
        pass
```

```
class LinearRegression(MLModel):
```

```
    def train(self):
```

```
        print("Training Linear Regression")
```

```
def predict(self):  
    print("Predicting Output")
```

```
model = LinearRegression()
```

```
model.train()
```

```
model.predict()
```

AI Example – Deep Learning

```
from abc import ABC, abstractmethod
```

```
class NeuralNetwork(ABC):
```

```
    @abstractmethod
```

```
    def forward(self):
```

```
        pass
```

```
class CNN(NeuralNetwork):
```

```
    def forward(self):
```

```
        print("Running CNN Forward Pass")
```

```
cnn = CNN()
```

```
cnn.forward()
```

AI Example – AI Assistant

```
from abc import ABC, abstractmethod
```

```
class AIAssistant(ABC):
```

```
    @abstractmethod
```

```
    def respond(self):
```

```
        pass
```

```
class ChatAssistant(AIAssistant):
```

```
    def respond(self):
```

```
        print("Generating AI Response")
```

```
assistant = ChatAssistant()
```

```
assistant.respond()
```

Real-World Example – Payment Gateway

```
from abc import ABC, abstractmethod
```

```
class Payment(ABC):
```

```
    @abstractmethod
```

```
    def pay(self):
```

```
        pass
```

```
class UPIPayment(Payment):
```

```
    def pay(self):
```

```
        print("UPI Payment Successful")
```

```
class CreditCardPayment(Payment):
```

```
    def pay(self):
```

```
        print("Credit Card Payment Successful")
```

```
upi = UPIPayment()
```

```
card = CreditCardPayment()
```

```
upi.pay()
```

card.pay()

Advantages of Abstraction

- Reduces software complexity.
 - Improves readability.
 - Makes code easier to maintain.
 - Promotes reusable code.
 - Encourages modular design.
 - Provides a standard interface.
 - Improves scalability.
 - Makes large applications easier to develop.
-

Applications of Abstraction

Abstraction is widely used in:

- Artificial Intelligence
- Machine Learning
- Deep Learning
- Banking Systems
- Hospital Management Systems
- ERP Software
- CRM Applications
- Cloud Computing
- Robotics
- Web Development
- Mobile Applications

Practical Exercise 1

Create an abstract class:

Shape

Abstract Method:

- `area()`

Create child classes:

- Circle
- Rectangle
- Triangle

Implement the `area()` method in each class.

Practical Exercise 2

Create an abstract class:

Vehicle

Methods:

- `start()`
- `stop()`

Create:

- Car
- Bike
- Bus

Implement both methods.

Practical Exercise 3

Create an abstract class:

AIModel

Methods:

- train()
- predict()

Create child classes:

- ClassificationModel
- RegressionModel
- ClusteringModel

Implement both methods.

Practical Exercise 4

Create an abstract class:

Notification

Method:

- send()

Create child classes:

- EmailNotification
- SMSNotification
- PushNotification

Implement the send() method.

Assignment

Smart AI Healthcare System

Create an abstract class:

HealthcareModel

Abstract Methods

- analyze()
- generate_report()

Child Classes

- DiseasePredictionModel
- MedicalImageAnalyzer
- PatientRiskAssessment

Requirements

- Implement all abstract methods.
 - Create objects for each model.
 - Display different outputs for each model.
 - Demonstrate how Abstraction provides a common interface while allowing different implementations.
-

Interview Questions

1. What is Abstraction?
2. Why is Abstraction important?
3. What is an Abstract Class?
4. What is an Abstract Method?
5. What is the purpose of the abc module?
6. What is the purpose of the ABC class?
7. What is the use of the @abstractmethod decorator?
8. Can we create an object of an Abstract Class? Why?
9. Differentiate between Abstraction and Encapsulation.
10. Explain a real-world example of Abstraction.

11. How is Abstraction used in AI and Machine Learning?

12. What happens if a child class does not implement all abstract methods?

Key Takeaways

- Abstraction is the **fourth and final pillar of OOP**.
- It focuses on **what an object does rather than how it does it**.
- Python implements abstraction using the **abc module**.
- **Abstract Classes** act as blueprints for other classes.
- **Abstract Methods** enforce a common interface for child classes.
- Abstraction is widely used in professional software, AI frameworks, machine learning libraries, and enterprise applications.

Next Class Preview

Class 17 – Exception Handling in Python

In the next session, students will learn:

- Introduction to Exceptions
- Types of Errors in Python
- try, except, else, and finally
- Raising Exceptions with raise
- Creating Custom Exceptions
- Exception Handling in AI & Machine Learning Applications
- Best Practices for Writing Robust Python Programs